Study Project — Phase 3

(Implementation Readiness & Validation)

Course Title:	Study Project
Project Title:	Operational Complexity Meter (OCM): Quantifying Operational Complexity in Distributed Systems
Group Number:	1
Student Name:	Aabid Ali Sofi
Student ID:	2023EBCS041
Advisor:	Preethy P Johny
Date of Submission:	March 2, 2026

Table of Contents

1. Introduction 3
   1.1. Purpose of Phase 3 3
   1.2. Summary of Work Completed 3
2. Implementation Overview 3
   2.1. Implementation Status 3
   2.2. Implemented Features 4
3. System Validation and Testing 5
   3.1. Testing Strategy 5
   3.2. Test Cases and Results 5
   3.3. Validation Summary 7
4. Performance and Reliability Analysis 7
5. Risk Analysis and Mitigation Review 8
   5.1. Identified Risks 8
   5.2. Mitigation Effectiveness 8
6. Limitations and Constraints 9
   6.1. Current Limitations 9
   6.2. Assumptions 9
   6.3. Constraints 9
7. Future Enhancements and Scope Extension 9
8. Learning Outcomes and Reflections 10
   8.1. Technical Skills Gained 10
   8.2. Design and Problem-Solving Insights 10
   8.3. Challenges and Lessons Learned 10
9. Final Deliverables 10
10. Conclusion 11
11. Supervisor Review and Approval 11

1. Introduction

1.1. Purpose of Phase 3

Phase 3 demonstrates that the Operational Complexity Meter (OCM) has reached full implementation readiness. The objectives of this phase are:

1. Demonstrating implementation readiness by delivering a feature-complete system that computes all six operational complexity metrics.
2. Validating design choices through automated testing, evidence traceability, and end-to-end pipeline verification.
3. Assessing system reliability, limitations, and future potential based on real-world testing against Kubernetes repositories.

1.2. Summary of Work Completed

Phase 1 — Problem Definition & Planning

- Identified the gap: no tool exists that quantifies *operational* complexity (deployment, configuration, dependency management) as distinct from code complexity.
- Defined the six metrics (CSA, DD, DB, CV, FE, CDR) and the composite OCM scoring model.
- Planned the single-binary Go architecture with embedded SQLite and web dashboard.

Phase 2 — Design & Proof of Concept

- Delivered a working PoC implementing two of six metrics (CSA and DD).
- Established the end-to-end pipeline: YAML parsing, metric computation, SQLite persistence, REST API, and web dashboard.
- Defined the database schema, functional requirements (FR1–FR10), and modular package structure.
- Validated feasibility with a demo against a sample Kubernetes repository.

2. Implementation Overview

2.1. Implementation Status

All modules planned during Phase 2 have been **fully implemented**. The system is feature-complete with respect to the six-metric scope defined in Phase 1.

Module	Description
`model`	Metric types, default weights (equal 1/6 each), data structures for services, metric points, scores, evidence, and analysis results.

`parser`	Kubernetes YAML parser — extracts CSA facts, dependency relationships (7 suffix patterns, hostname:port regex, `.svc.cluster.local` FQDN), and Failure Exposure signals (5 detection rules).
`pipeline`	Full orchestration — discovery, service identification, all six metric computations, min-max normalization, and weighted composite scoring.
`gitlog`	Git history extraction for Change Volatility — parses `git log` output, filters by config file extensions, maps files to services.
`storage`	SQLite persistence — schema migration, transactional run saves, time-series queries, evidence retrieval.
`api`	REST API — 6 endpoints for health, services, overview, score series, metric series, and evidence. CORS middleware.
`dashboard`	Embedded web UI — `go:embed` static assets, Stripe-inspired dark theme, 4 canvas-based chart types, responsive layout.
`cmd/ocm`	CLI entry point — 7 flags, orchestrates the full startup sequence.

Partially or Unimplemented: None. All planned components are complete.

2.2. Implemented Features

Core Metric Computation (6 metrics):

1. **CSA (Configuration Surface Area)** — Count of configurable knobs: environment variables, container ports, resource limits/requests, replicas, and spec-level keys.
2. **DD (Dependency Depth)** — Longest path in the SCC-condensed dependency DAG, computed using Tarjan's algorithm for deterministic cycle handling.
3. **DB (Dependency Breadth)** — Sum of in-degree and out-degree in the service dependency graph.
4. **CV (Change Volatility)** — Git commits touching service configuration files within a configurable time window (default 30 days).
5. **FE (Failure Exposure)** — Externally exposed endpoints detected via 5 rules: LoadBalancer/NodePort services, Ingress resources, hostPort bindings, and external URLs in environment variables.
6. **CDR (Configuration Drift Risk)** — Environment-specific overrides counted across recognized environment directories (dev, staging, prod, qa, test, uat, preview, canary, local).

Composite Scoring:

- Min-max normalization per metric across the service cohort:

$$\text{norm} = \frac{v - \min}{\max - \min}$$

- When max = min, the normalized value defaults to 0.
- Composite OCM score:

$$\text{OCM} = \sum_{m \in M} w_m \cdot \text{norm}_m \quad \text{where } w_m = 1/6$$

Data Handling and Persistence:

- SQLite database with 4 tables and 3 indexes, foreign key enforcement.
- Transactional writes — each analysis run is persisted atomically.
- Time-series storage for trend analysis across multiple runs.
- Evidence items linked to specific metrics, services, and source files.

User Interaction Flows:

- CLI with 7 configurable flags for repository path, database, host/port, service key strategy, CV window, and JSON output.
- Web dashboard with two views: **Overview** (repo-wide aggregates) and **Services** (comparison table).
- Per-service detail: metric tiles with sparklines, trend chart, radar chart, bar chart.
- Evidence drill-down modal showing Component, Key, Value, Manifest, and Source columns.
- Responsive layout with mobile sidebar toggle.

Integration:

- Git integration for CV metric (best-effort — gracefully degrades when Git is unavailable).
- Filesystem-based environment detection for CDR metric.
- CORS middleware for cross-origin browser access.

3. System Validation and Testing

3.1. Testing Strategy

The project employs a layered testing approach:

- **Unit testing:** Isolated tests for individual functions (normalization, dependency extraction, git log parsing, service name normalization).
- **Integration testing:** End-to-end pipeline tests that create temporary Kubernetes YAML repositories, run the full analysis pipeline, and verify all six metrics are computed with correct values.
- **Persistence testing:** Round-trip tests that save analysis runs to an in-memory SQLite database and verify retrieval of all metric types and evidence items.

All tests use Go's standard `testing` package and can be run with `go test ./...`.

3.2. Test Cases and Results

The test suite consists of **32 test functions** across 4 packages.

Parser Tests (16 tests)

Test	Expected Behavior	Observed	Status
FE: Service LoadBalancer	2 ports -> 2 exposed endpoints	2 endpoints	Pass
FE: Ingress	2 rules, 3 paths -> 3 endpoints	3 endpoints	Pass
FE: ExternalName	1 external integration	1 integration	Pass

FE: External URL	Only external URLs counted	1 external URL	Pass
FE: NodePort	1 port -> 1 endpoint	1 endpoint	Pass
FE: ClusterIP	No exposure	0 endpoints	Pass
Dep: Service suffix	`DB_SERVICE=postgres` -> dep	[postgres, redis]	Pass
Dep: Addr suffix	Google Boutique pattern	Correct deps	Pass
Dep: Host suffix	`DB_HOST=postgres:5432`	[postgres, redis]	Pass
Dep: URL suffix	`API_URL=http://api:8080`	[api-gateway]	Pass
Dep: Hostname:port	Bare `mongo=user-db:27017`	[user-db]	Pass
Dep: Cluster FQDN	`.svc.cluster.local`	[zipkin]	Pass
Dep: Plain strings	No deps from plain values	0 dependencies	Pass
Dep: Numeric rejected	Port numbers not deps	1 dep only	Pass
Normalize: Numeric	`"5432" -> "", "redis" -> "redis"`	Correct	Pass
Dep: CronJob	CronJob containers parsed	Correct deps	Pass

Pipeline Tests (8 tests)

Test	Expected Behavior	Observed	Status
DD: Cycle handling	SCC condensation: ab cycle	Correct DD values	Pass
Normalize: max=min	Same values -> 0	Both normalize to 0	Pass
Pipeline: Run smoke	Produces valid score	Score in [0,1]	Pass
DB: In/out degree	a->{b,c}, b->{c}	Correct degrees	Pass
DB: No dependencies	Isolated services	db = 0 for both	Pass
DB: Self-dep ignored	a->{a,b}, self-dep skipped	Correct values	Pass
All 6 metrics	Temp repo with 2 services	All present	Pass
All metrics in composite	Manual data, full pipeline	Values in [0,1]	Pass

Git Log Tests (6 tests)

Test	Expected Behavior	Observed	Status
Multiple commits	2 commits parsed correctly	Correct	Pass
Empty output	0 commits returned	Empty slice	Pass
Single commit, no files	1 commit, 0 files	Correct	Pass
Malformed timestamp	Hash parsed, time is zero	Graceful degradation	Pass

Hash only	Hash parsed without timestamp	Correct	Pass
Options defaults	30-day window, extensions set	Defaults applied	Pass

Storage Tests (2 tests)

Test	Expected Behavior	Observed	Status
Save/retrieve all metrics	Round-trip for all 6 types	All verified	Pass
Empty source path	DD/DB/CV/CDR evidence saves	Persisted	Pass

3.3. Validation Summary

- **Requirements coverage:** All 10 functional requirements from Phase 2 (FR1–FR10) are satisfied. Additionally, four new metrics (DB, CV, FE, CDR) extend the system beyond the original MVP scope.
- **Evidence traceability:** Every metric value is backed by queryable evidence items that trace back to specific YAML files, manifest kinds, and configuration keys.
- **Deviations:** No deviations from expected behavior. The system handles edge cases (cycles in dependency graphs, missing Git, numeric values, CronJob manifests) as designed.

4. Performance and Reliability Analysis

Aspect	Details
Responsiveness	Analysis of small-to-medium repositories (10–50 services) completes in under 5 seconds. API responses and dashboard rendering are sub-second.
Stability	Single Go binary with no goroutine leaks. SQLite transactions ensure atomic writes. HTTP server uses 5-second `ReadHeaderTimeout`.
Resource Usage	Minimal memory footprint — YAML files parsed one document at a time. SQLite grows 50KB per run for a 10-service repo. Binary size 15MB.
Bottlenecks	The `git log` subprocess is the slowest component for large histories. The `--cv-window` flag mitigates this. CDR directory heuristics may produce false positives for unconventional structures.

5. Risk Analysis and Mitigation Review

5.1. Identified Risks

Risk	Category	Description
R1	Service identification	Different directory structures may yield inconsistent service names.
R2	Dependency extraction	Heuristic-based detection may miss or falsely identify dependencies.
R3	Metric validity	Heuristic-based metrics may not accurately reflect true complexity.
R4	Scope creep	Expanding beyond the six-metric scope during implementation.
R5	External dependency	CV metric depends on Git being available in the environment.
R6	Scalability	Memory and time costs for repositories with hundreds of services.

5.2. Mitigation Effectiveness

Risk	Mitigation Applied	Effectiveness	Status
R1	Two strategies (`dir` and `manifest`) configurable via `--service-key`.	Effective	Mitigated
R2	Seven suffix patterns, hostname:port regex, FQDN detection, CronJob support. 16 parser tests.	Effective	Mitigated
R3	Full evidence traceability — every metric value links to specific YAML keys and files.	Effective	Mitigated
R4	Strict adherence to the six-metric model.	Fully effective	Mitigated
R5	Best-effort: if `git` is not in PATH, CV returns zero and pipeline continues.	Effective	Mitigated
R6	Sequential file processing avoids memory spikes. Not tested at extreme scale (500+).	Partial	Remaining

6. Limitations and Constraints

6.1. Current Limitations

Limitation	Details
No Helm support	The parser operates on raw YAML. Helm charts must be rendered (`helm template`) before analysis.
Heuristic detection	Dependencies inferred from env var names/values. Mechanisms not reflected in YAML (service mesh, external MQs) are not detected.
Single-run model	Analysis runs once on startup. No daemon mode, scheduled execution, or file-watching.
Localhost only	No authentication or authorization. Dashboard and API intended for local use only.
No multi-repo	Each run analyzes a single repository path.

6.2. Assumptions

- Kubernetes manifests follow standard API conventions (Deployment, Service, Ingress, etc.).
- Service names are stable and deterministic based on directory structure or manifest metadata.
- Git history is available in the analyzed repository for CV metric computation.
- Environment directories follow common naming conventions (dev, staging, prod, etc.).

6.3. Constraints

- **Time:** Study project timeline limited advanced features such as CI/CD integration and multi-user access.
- **Technology:** Pure-Go SQLite driver trades some performance for zero CGO dependency.
- **Scope:** Only Kubernetes YAML is supported. Docker Compose, Terraform, and other IaC formats are excluded.

7. Future Enhancements and Scope Extension

Category	Enhancements
Feature	Helm chart support, watch mode, custom metric weights via UI, multi-repository analysis, CI/CD integration with complexity thresholds.
Performance	Incremental analysis (recompute only affected services), parallel YAML parsing with goroutines, database compaction for old time-series data.
Scalability	Remote database support (PostgreSQL), token-based authentication for non-localhost usage.

Additional Use Cases	Docker Compose and Terraform support, alerting on OCM threshold violations, historical diff view, PDF/CSV report export.

8. Learning Outcomes and Reflections

8.1. Technical Skills Gained

- Designing and implementing a metrics engine with evidence traceability in Go.
- Working with graph algorithms (Tarjan's SCC, DAG longest path) for real-world dependency analysis.
- Building a complete application with embedded database, REST API, and web dashboard in a single binary.
- Implementing canvas-based data visualizations (radar chart, sparklines, trend lines) without third-party libraries.
- Writing comprehensive tests for heuristic-based systems with numerous edge cases.

8.2. Design and Problem-Solving Insights

- **Evidence-first design:** Attaching evidence to every metric value proved essential for user trust. A complexity score is meaningless without the ability to explain *why* a service scored high.
- **Heuristic boundaries:** Dependency detection heuristics are imperfect, but combining multiple strategies with transparent evidence makes the system useful despite limitations.
- **Min-max normalization:** When all services share the same metric value, the normalized value defaults to 0 — a deliberate choice to avoid a single-service repo appearing "maximally complex."

8.3. Challenges and Lessons Learned

- **YAML parsing:** Kubernetes manifests vary significantly across resource kinds. Supporting all common kinds required careful recursive traversal.
- **Cycle handling:** Circular dependencies initially caused infinite loops during DD computation. Tarjan's SCC algorithm resolved this deterministically.
- **Git integration:** Parsing `git log` output is fragile. Structured `--format` flags and best-effort approach ensure functionality without Git.
- **Dashboard without frameworks:** Vanilla Canvas and CSS required more effort than a framework-based approach but resulted in zero frontend build dependencies.

9. Final Deliverables

Deliverable	Description
Source Code	Complete Go codebase — 12 source files (3,800 lines) across 7 packages.

Compiled Binary	`ocm` — single executable, no external runtime dependencies.
Test Suite	32 test functions covering parser, pipeline, git log, and storage. Run with `go test ./...`.
SQLite Database	`ocm.sqlite` — analysis results for time-series tracking.
Web Dashboard	Embedded interactive dashboard with overview, service detail, charts, and evidence drill-down.
REST API	6 endpoints serving JSON data for services, metrics, scores, and evidence.
Specifications	8 specification files in `specs/` covering all system modules.
Phase Documents	Phase 2 design document and this Phase 3 document.
Presentation	Polylux/Typst presentation (`docs/presentation.pdf`).
Project Repository	https://github.com/aabidsofi19/ocm

10. Conclusion

The Operational Complexity Meter (OCM) has been fully implemented as a single-binary Go tool that quantifies operational complexity in distributed systems deployed on Kubernetes. All six planned metrics — Configuration Surface Area, Dependency Depth, Dependency Breadth, Change Volatility, Failure Exposure, and Configuration Drift Risk — are computed, normalized, and combined into a composite OCM score.

The system is validated by 32 automated tests covering metric computation, dependency graph algorithms, Git integration, and database persistence. Every metric value is backed by evidence items that trace to specific YAML files, manifest keys, and configuration parameters, ensuring full explainability.

The project has progressed from problem definition (Phase 1) through design and proof-of-concept (Phase 2) to a feature-complete implementation (Phase 3) that meets all defined functional and non-functional requirements. The modular architecture supports future extension to additional IaC formats, custom metric weights, and team-wide deployment.

The system is ready for evaluation and demonstration.

11. Supervisor Review and Approval

Advisor Feedback:

Supervisor Comments:

Recommendations:

Signature: _____

Date: _____